14. Windows Packaging

Detailed beginner handbook - English and Arabic

Technologies: PyInstaller, Inno Setup

What you will learn

Bundle a small Python app and understand an installer workflow.

PyInstaller bundles Python and required files; Inno Setup installs the bundle for Windows users.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولا، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/14-windows-packaging.
3. Read README.md.
4. Run pip install pyinstaller; pyinstaller --windowed app.py.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat PyInstaller, Inno Setup as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - Why packaging differs from simply running source code

What is it?

Source code assumes Python, packages, and assets exist. Packaging collects a controlled runtime so users can start the application without a development environment.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Development depends on installed Python and packages.
# A package includes the interpreter, imports, binaries, and application assets.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Why packaging differs from simply running source code.

هذا الدرس يشرح: Why packaging differs from simply running source code. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Virtual environments, locked versions, and reproducible builds

What is it?

Build from a clean virtual environment with pinned versions. Record the toolchain and regenerate dependencies so a future release can be reproduced.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip freeze > build-versions.txt
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Virtual environments, locked versions, and reproducible builds.

هذا الدرس يشرح: Virtual environments, locked versions, and reproducible builds. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - PyInstaller analysis, hidden imports, data files, and hooks

What is it?

PyInstaller analyses imports but dynamic imports may need hidden-import entries. add-data bundles frontend files and icons; hooks collect package-specific binaries and metadata.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
python -m PyInstaller --onedir --windowed --name LearningApp `
 --add-data "frontend/dist;frontend/dist" app.py
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates PyInstaller analysis, hidden imports, data files, and hooks.

هذا الدرس يشرح: PyInstaller analysis, hidden imports, data files, and hooks. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - One-folder versus one-file packaging trade-offs

What is it?

One-folder builds start faster and are easier to inspect; one-file builds extract at launch and may trigger more security scanning. Test the chosen trade-off.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# One-folder: dist/LearningApp/ with visible files, faster startup.
# One-file: one executable that extracts at startup.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates One-folder versus one-file packaging trade-offs.

هذا الدرس يشرح: One-folder versus one-file packaging trade-offs. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - Windowed applications, icons, manifests, and version resources

What is it?

--windowed hides the console for GUI apps. Provide a real .ico, version metadata, application manifest, and an accessible way to retrieve diagnostic logs.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
python -m PyInstaller --windowed --icon app.ico app.py
# Add Windows version resources and keep diagnostic logs accessible.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Windowed applications, icons, manifests, and version resources.

هذا الدرس يشرح: Windowed applications, icons, manifests, and version resources. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Testing the dist folder on a clean Windows machine

What is it?

Copy dist to a clean Windows account or virtual machine without Python. Test startup, upload, charts, exports, dialogs, updates, uninstall, and offline behaviour.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
& '.\dist\LearningApp\LearningApp.exe'
# Test on a clean Windows VM: launch, dialogs, export, close, relaunch.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Testing the dist folder on a clean Windows machine.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Testing the dist folder on a clean Windows machine هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Inno Setup sections, files, shortcuts, uninstall, and upgrades

What is it?

Inno Setup's Setup section defines identity and install location; Files copies the bundle; Icons creates shortcuts; uninstall information and upgrade rules support lifecycle management.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
[Setup]
AppName=LearningApp
AppVersion=1.0.0
DefaultDirName={autopf}\LearningApp
[Files]
Source: "dist\LearningApp\*"; DestDir: "{app}"; Flags: recursesubdirs
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Inno Setup sections, files, shortcuts, uninstall, and upgrades.

هذا الدرس يشرح: Inno Setup sections, files, shortcuts, uninstall, and upgrades. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Application versioning and safe update manifests

What is it?

Use semantic versions consistently in the app, installer, and manifest. An updater should verify trusted URLs, signatures or hashes, download safely, and support recovery.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
APP_VERSION = '1.2.0'
# Use the same version in app, installer, filename, and update manifest.
# Verify update URL, hash/signature, and rollback path.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Application versioning and safe update manifests.

هذا الدرس يشرح: Application versioning and safe update manifests. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Code signing, hashes, release integrity, and SmartScreen

What is it?

Code signing proves publisher identity and package integrity. Timestamp signatures, publish cryptographic hashes, protect signing keys, and understand that reputation develops over releases.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
Get-FileHash .\LearningApp-Setup.exe -Algorithm SHA256
# Sign with a protected certificate and timestamp; verify signature before release.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Code signing, hashes, release integrity, and SmartScreen.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Code signing, hashes, release integrity, and SmartScreen هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Release checklists, rollback, support logs, and automation

What is it?

Automate clean build, tests, package, malware scan, signature verification, install/uninstall smoke tests, checksum publication, rollback retention, and release notes.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Windows Packaging, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Release pipeline:
# clean -> test -> build -> package -> scan -> sign -> install test -> hash -> publish -> retain rollback
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Release checklists, rollback, support logs, and automation.

هذا الدرس يشرح: Release checklists, rollback, support logs, and automation. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Bundle a small Python app and understand an installer workflow.

Starting example

```
pyinstaller --windowed --onedir --name LearningApp app.py
ISCC installer.iss
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines PyInstaller, Inno Setup with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة آمنة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step	You completed the learning path.